A arquitetura Transformer

Transformers e Large Language Models

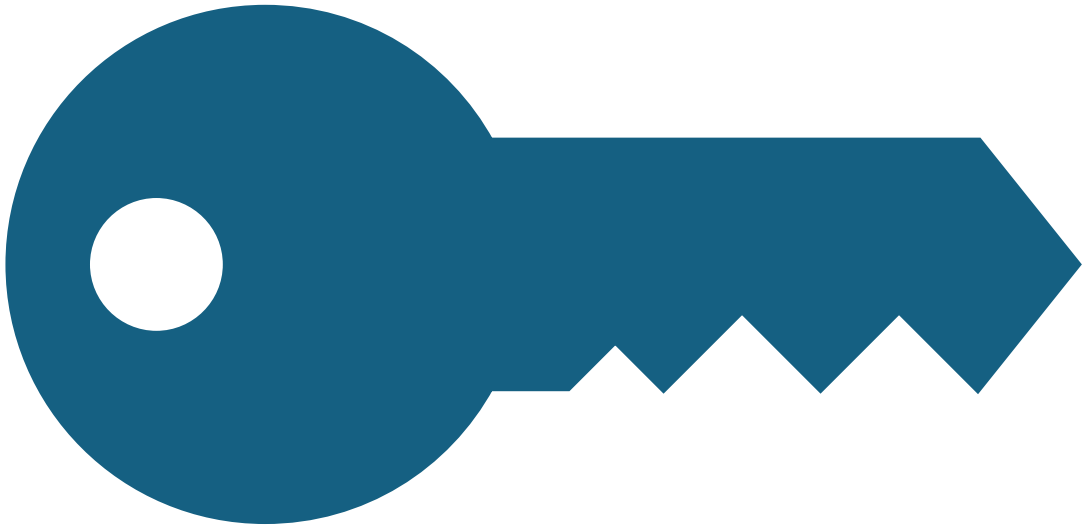
Aula 03 – 3 de Abril de 2026

Vítor Wilher

Data Tech Lead na Análise Macro

Plano de Voo

- A Arquitetura Transformer.
- Exemplo Prático de como a arquitetura Transformer funciona;

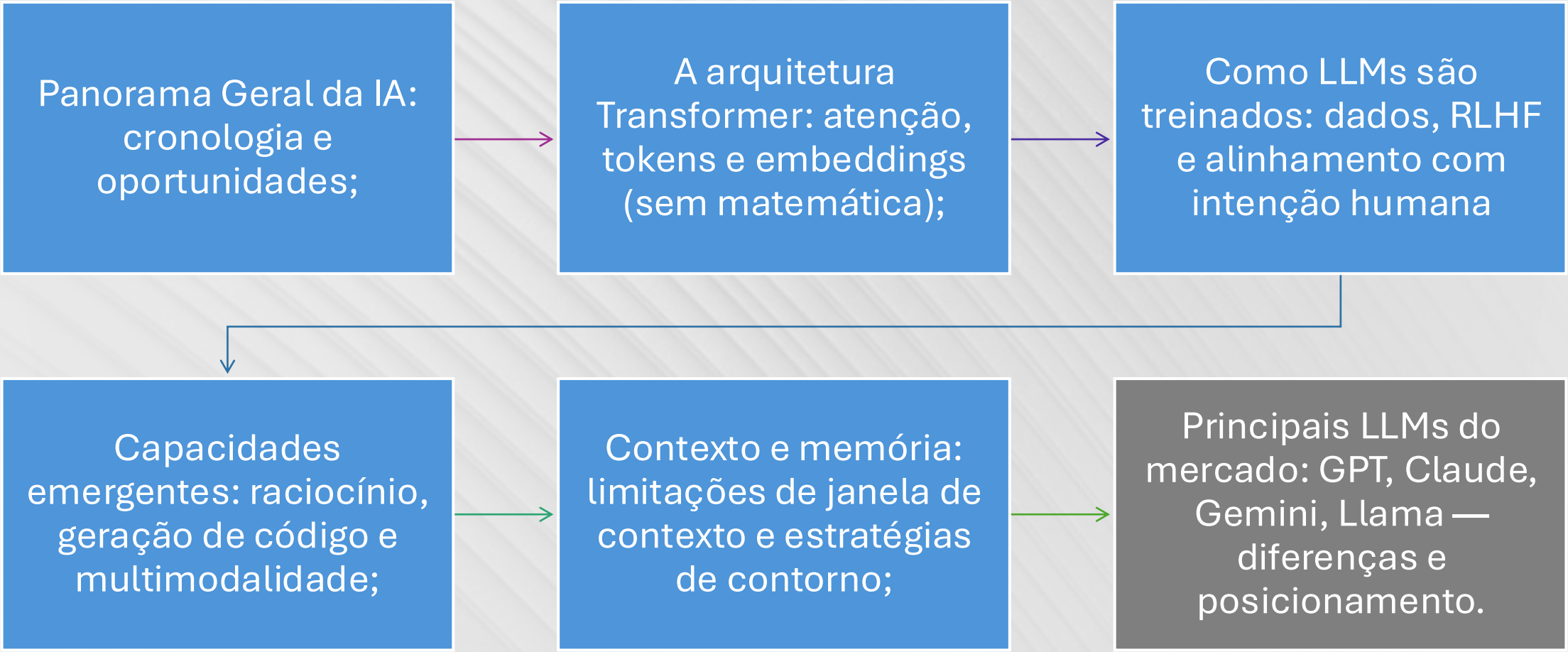


Ementa Resumida do Curso

Detalhar a arquitetura *Transformer* e os *LLMs* — a tecnologia específica que impulsionou a revolução da **IA generativa** em linguagem.

Compreender como esses modelos funcionam é essencial para usá-los estrategicamente.

O que veremos hoje...



O que é o Transformer?

O Transformer foi introduzido em 2017 por pesquisadores do Google no famoso artigo **“Attention Is All You Need”**. Sua proposta era a de substituir completamente as camadas de recorrência (o “coração” das LSTMs) por um mecanismo chamado **auto-atenção (*self-attention*)**. A arquitetura foi originalmente projetada para tradução automática e é composta por duas partes principais: o Encoder e o Decoder.

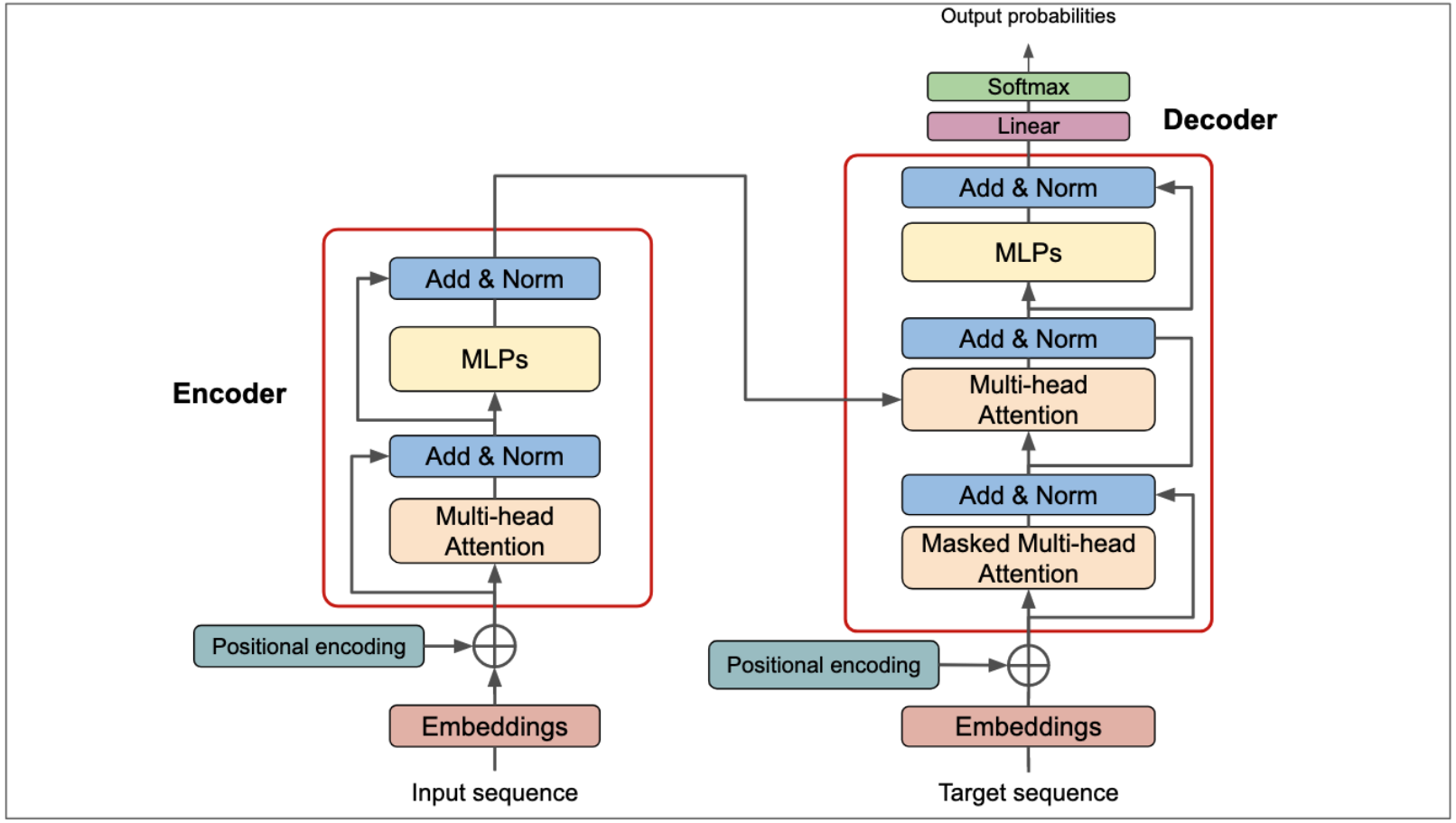
Encoder (Codificador)

Pense no Encoder como o “leitor” do modelo. Sua função é processar toda a sequência de entrada (seja uma frase em português ou outra língua) e construir uma representação numérica rica em contexto para cada ponto da sequência. Ele lê a frase inteira para entender as relações entre todas as palavras antes de passar a informação adiante.

Decoder (Decodificador)

O Decoder é o “escritor”. Ele recebe a representação contextualizada do Encoder e gera a sequência de saída, um item de cada vez. No caso da tradução, ele gera a frase em outro idioma, palavra por palavra.

Encoder-Decoder



Entendendo a arquitetura...

- O que podemos observar pela imagem é que o processo se baseia, principalmente, em redes neurais (MLPs: Multi-layer Perceptrons), responsáveis por treinar e gerar parâmetros que permitem ao modelo compreender as dinâmicas da linguagem.

- Mas surge a pergunta: como uma máquina consegue entender e gerar linguagem?

- Como ela “aprende” o significado das palavras e entende a estrutura sintática usada pelos humanos?

- Todo esse funcionamento é explicado pelos componentes presentes dentro dos blocos de Encoder e Decoder, que serão detalhados nos próximos tópicos.

Entendendo a arquitetura...

- Um ponto importante a se destacar é que a arquitetura mostrada na figura representa o Transformer original, proposto no artigo *Attention Is All You Need* (Vaswani et al., 2017).

- Nesse trabalho, os autores buscavam uma estrutura voltada para tarefas de tradução automática, e por isso adotaram o formato **encoder-decoder**.

- No entanto, essa arquitetura pode ser adaptada, dando origem a modelos compostos apenas por **encoders** ou apenas por **decoders**, cada um com objetivos e aplicações diferentes.

Entendendo a arquitetura...

- A arquitetura **somente de encoder** é ideal para tarefas de compreensão e análise de texto, nas quais o modelo precisa entender o contexto completo de uma sequência. É usada em modelos como o **BERT**, aplicados em classificação, análise de sentimento, *named entity recognition* e recuperação de informação.

- A arquitetura **somente de decoder** é voltada para tarefas de geração de texto, em que o modelo precisa prever o próximo token com base nos anteriores. É usada em modelos como o **GPT**, aplicados em tradução, resumo e geração autônoma de linguagem.

- Já a arquitetura **encoder-decoder** combina o melhor dos dois mundos: o encoder entende o significado da entrada, enquanto o decoder gera a saída com base nessa compreensão. É o formato típico de modelos de tradução e sumarização, como o **T5** e o **BART**.

Entendendo a arquitetura...

- Esses modelos são compostos por uma **série de blocos Transformer**, que se repetem diversas vezes — desde cerca de seis blocos no artigo original de Vaswani et al. (2017), até mais de cem em grandes modelos modernos.

- Cada bloco recebe uma entrada, realiza o processamento das informações e passa o resultado para o próximo bloco, formando uma cadeia de camadas empilhadas.

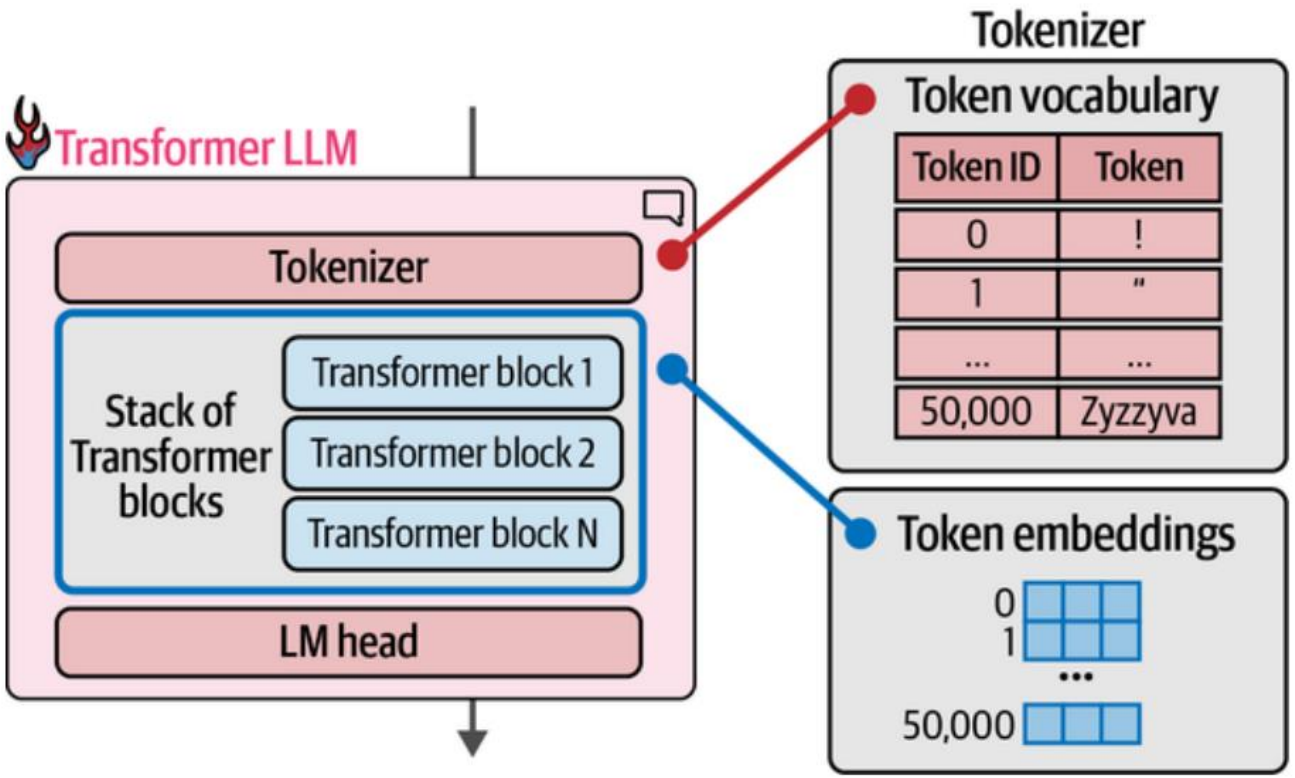
Entendendo a arquitetura em linhas gerais

Tokenização e criação de embeddings

- A **primeira etapa** do processo envolve a **tokenização e a criação dos embeddings**.

- Nessa fase, o texto é dividido em pequenas unidades chamadas *tokens* (palavras ou pedaços de palavras), que são convertidas em vetores numéricos.

- Esse passo é fundamental para que a máquina consiga “ler” e representar a linguagem humana em formato matemático.

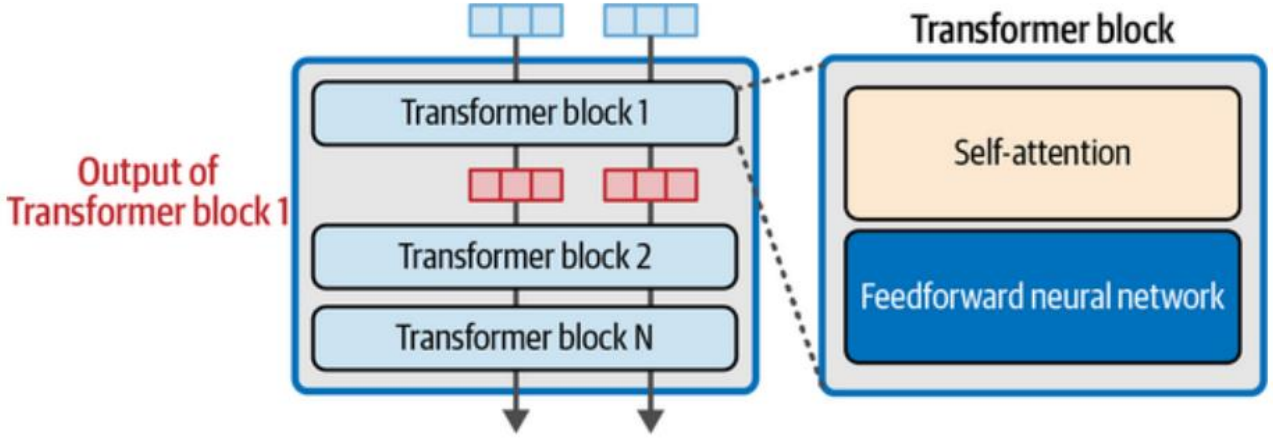


Atenção e Feedforward

- Em seguida, cada bloco Transformer é composto por dois componentes principais:

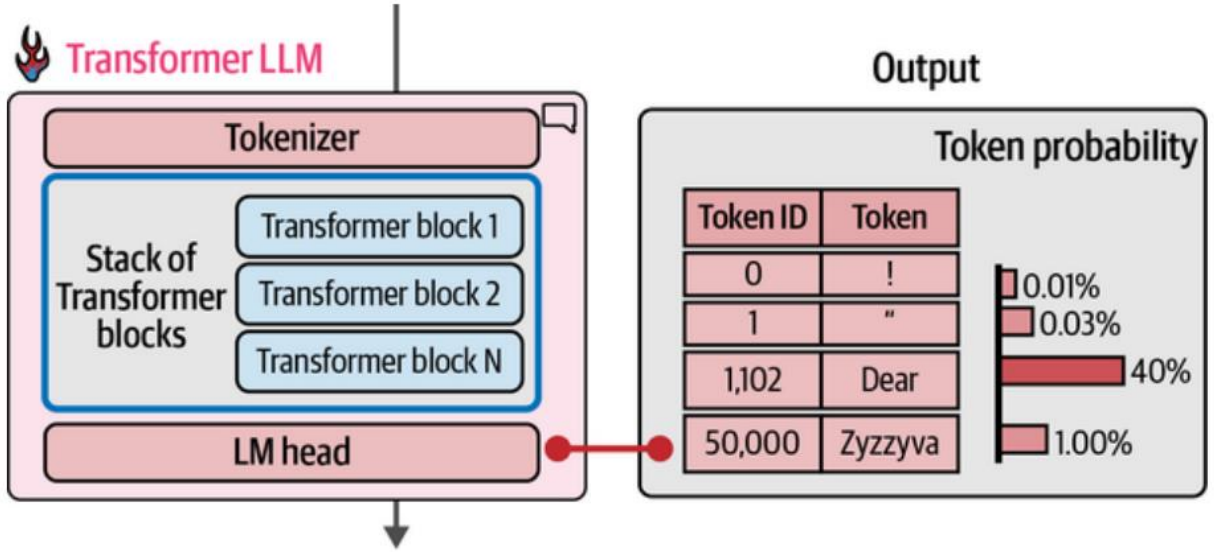
- **Camada de Atenção (Attention Layer)**
Essa camada é responsável por identificar e incorporar **informações relevantes** de outros tokens na sequência. Em outras palavras, ela permite que o modelo entenda **quais palavras estão relacionadas entre si**, mesmo que distantes no texto. É nessa etapa que o modelo "presta atenção" nas partes mais importantes do contexto.

- **Camada Feedforward (Feedforward Layer ou MLPs)**
Após o mecanismo de atenção, os vetores passam por uma rede neural do tipo **MLP (Multi-Layer Perceptron)**. Essa é a parte onde ocorre a **maior capacidade de processamento do modelo**, responsável por combinar, transformar e refinar as representações numéricas criadas pela atenção.



LM Head

- Por fim, após passarem por todos os blocos e suas transformações, os **vetores numéricos finais (output vectors)** são enviados ao **LM Head (Language Model Head)** — uma camada de saída que converte esses vetores em probabilidades sobre o próximo token da sequência, gerando assim o texto final.



O que cada bloco faz...

1. Tokenização e Embedding

- Tudo começa com a **entrada de dados (input sequence)**, seja um texto, uma sequência de preços ou um áudio. O modelo precisa converter isso em algo que ele entenda — números.

- **Tokenização** é o primeiro passo. No caso de textos, o modelo quebra as frases em pequenas unidades chamadas **tokens** — que podem ser palavras, subpalavras ou até caracteres.

- **Embedding** vem em seguida. Cada token é transformado em um vetor numérico de alta dimensão, o chamado **embedding**. Esse vetor captura as relações semânticas (no caso de texto) Por exemplo, embeddings próximos no espaço vetorial indicam tokens que possuem significados semelhantes ou comportamentos parecidos.

2. Self-Attention

- Depois que cada token é convertido em um vetor, o modelo precisa entender como cada um se relaciona com os outros. É nesse ponto que entra o **mecanismo de atenção (attention mechanism)** — o verdadeiro núcleo do Transformer.

- O **self-attention** permite que o modelo observe todos os tokens de uma sequência simultaneamente e identifique quais são mais importantes para compreender o contexto de cada um.

O mecanismo de atenção

O funcionamento da atenção envolve duas etapas principais:

- **Atribuir pesos de relevância** a cada token anterior em relação ao token atual;
- **Combinar as informações** ponderadas desses tokens para formar um novo vetor de saída.

Para ampliar essa capacidade, o Transformer executa várias atenções em paralelo — um processo chamado de **multi-head attention**.

Cada *head* aprende a capturar padrões diferentes do texto, como dependências gramaticais, relações semânticas ou referências a longo prazo. Isso permite que o modelo compreenda aspectos distintos da mesma sequência simultaneamente.

O mecanismo de atenção

Dentro de uma *attention head*, o cálculo começa com a criação de três vetores derivados de cada *embedding*:

- **Query (Q)** – representa o token que “pergunta” quais outros tokens são relevantes para ele;
- **Key (K)** – descreve o conteúdo de cada token, como se dissesse “eu represento esta informação”;
- **Value (V)** – contém a informação que será usada caso a *Key* seja considerada relevante.

3. Positional Encoding

- Um problema do self-attention é que ele não entende a ordem natural dos tokens. Para o modelo, “o gato dorme” e “dorme o gato” são tecnicamente a mesma coisa, pois a relação de atenção entre as palavras é a mesma. Isso acontece porque o mecanismo de atenção, por si só, é **invariante à posição** — ele apenas mede relações entre tokens, sem saber onde cada um está na sequência.

- Para resolver esse problema, o Transformer adiciona o **Positional Encoding (PE)**, um vetor que representa a **posição** de cada token na sequência. Esses vetores são somados diretamente aos embeddings das palavras antes de entrarem no mecanismo de atenção, de modo que cada posição carregue uma assinatura numérica única.

4. Feed-Forward Network

- Após as camadas de atenção e codificação posicional, cada token passa por uma **rede neural feed-forward**, aplicada individualmente a cada posição.

- Essa etapa é responsável por transformar as representações produzidas pela atenção em **informações mais abstratas e não lineares**. Ela ajuda o modelo a captar padrões complexos e a ajustar as relações aprendidas nas camadas anteriores.

- É como se, depois de olhar o contexto global, o modelo refinasse localmente o entendimento de cada token — adicionando nuances e captando interações sutis.

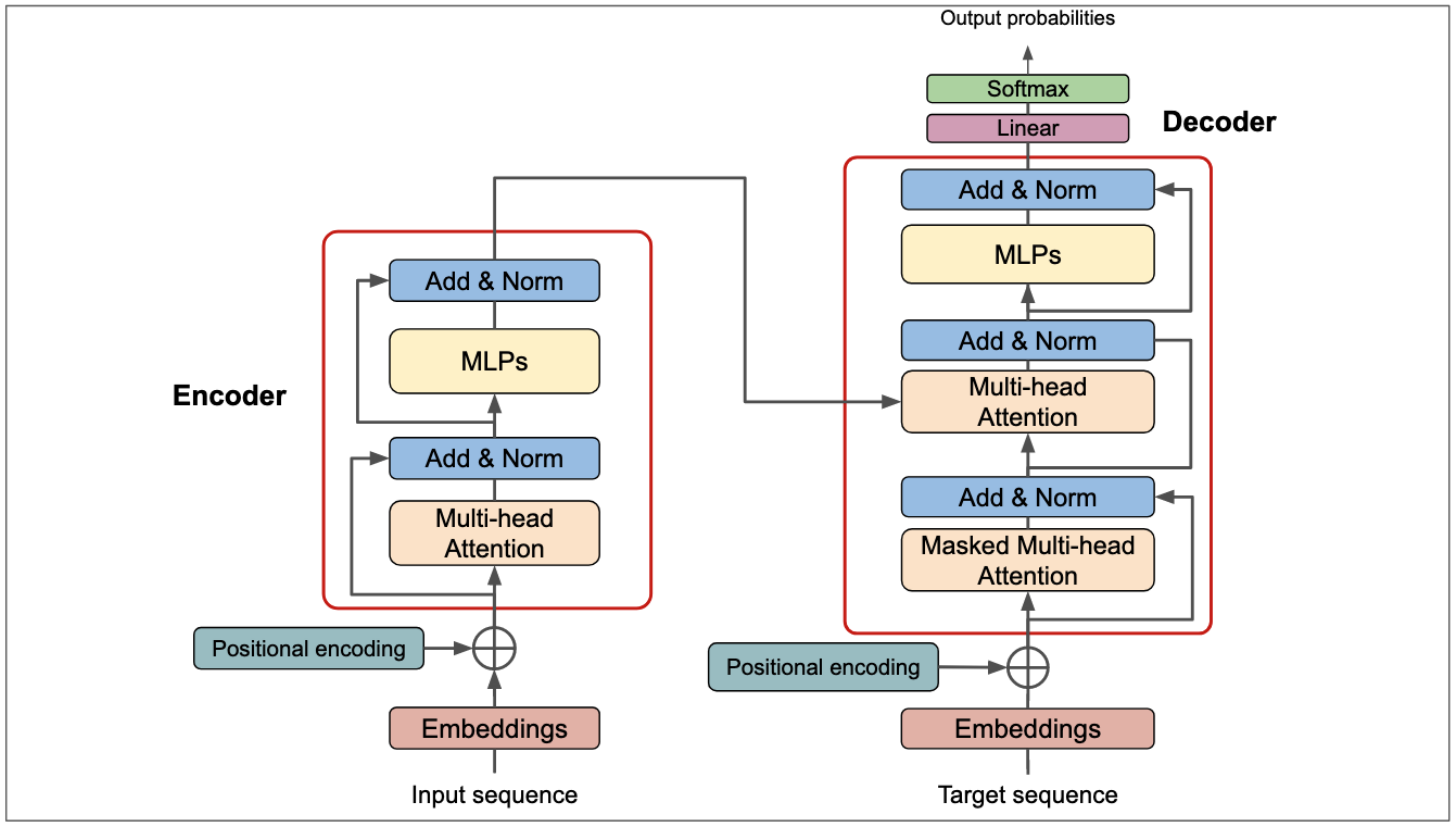
5. Multi-Head Attention: múltiplas perspectivas, um mesmo contexto

Fazer o processo de atenção uma única vez já permite que o modelo entenda como as palavras se relacionam entre si. No entanto, a linguagem — e o mundo — são complexos demais para serem compreendidos por apenas uma "visão". É por isso que o Transformer usa o conceito de **múltiplas cabeças de atenção** (*multi-head attention*).

- Cada cabeça funciona como um especialista observando a mesma frase sob um ângulo diferente.
  Por exemplo:
- uma cabeça pode focar em relações **de curto prazo**, como "o gato" → "dorme";
- outra pode buscar relações **de longo prazo**, como "ela" → "Maria", mesmo que as palavras estejam distantes na frase.

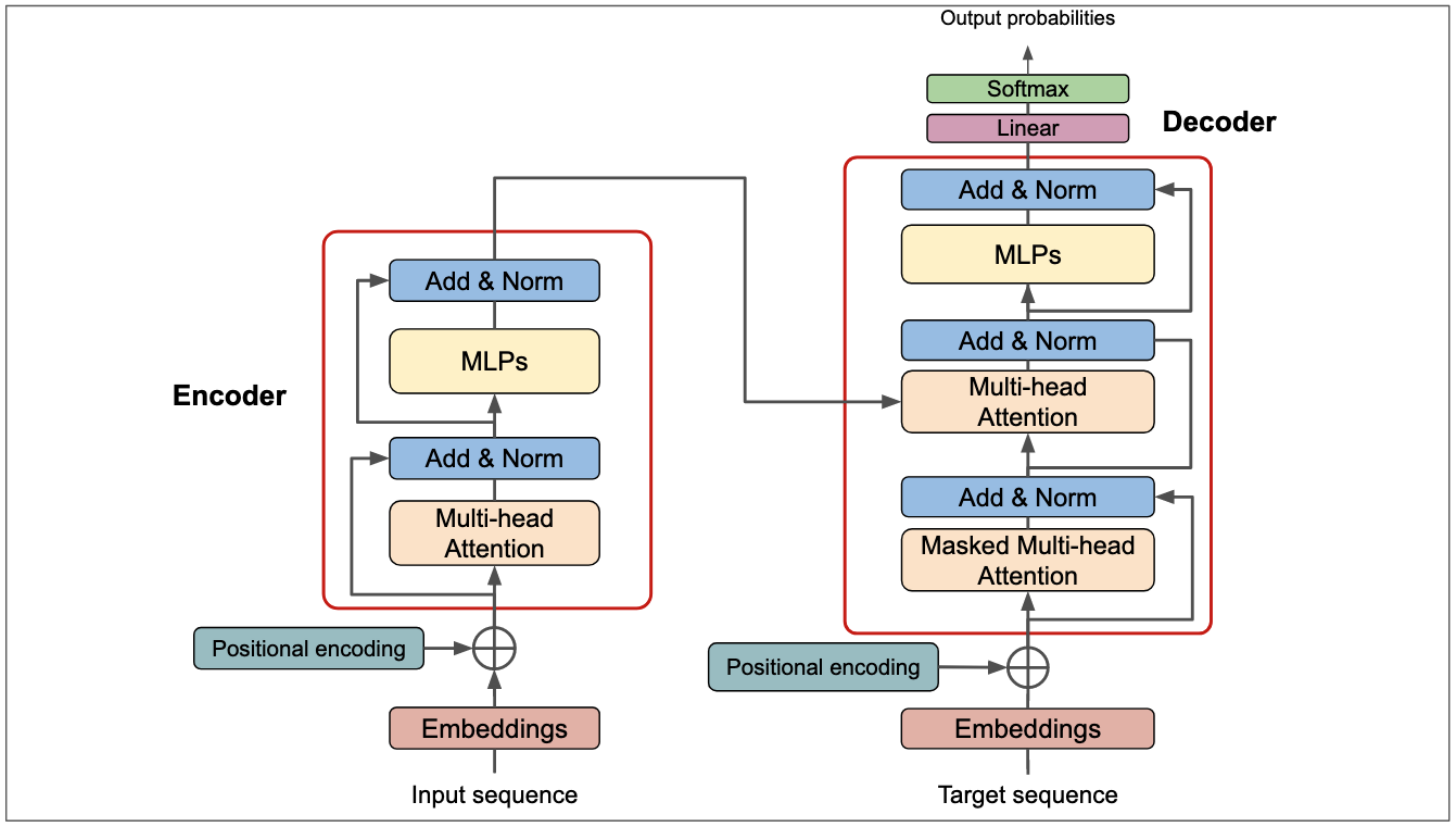
O fluxo de dados no Transformer

- O Lado do Encoder (O Leitor)
- A função do Encoder é ler a sequência de entrada e criar uma representação rica em contexto.
- **Entrada:** A *Input sequence* é convertida em **Embeddings** e somada ao **Positional Encoding** para adicionar o contexto de ordem.
- **Atenção e Processamento:** O resultado passa por um bloco que contém:
    - Uma camada de **Multi-Head Attention** para que os tokens troquem informações entre si.
    - Uma camada **Add & Norm**. O "Add" é uma **conexão residual** (soma da entrada com a saída da atenção) que estabiliza o treinamento. O "Norm" é uma **normalização de camada** que mantém os valores em uma escala controlada.
    - Uma rede **Feed-Forward (MLPs)** para processamento adicional.
    - Outra camada **Add & Norm** para finalizar o bloco.
- Este bloco pode ser empilhado várias vezes. A saída final do Encoder é uma memória contextualizada de toda a entrada, que é enviada ao Decoder na forma dos vetores **Key (K)** e **Value (V)**.



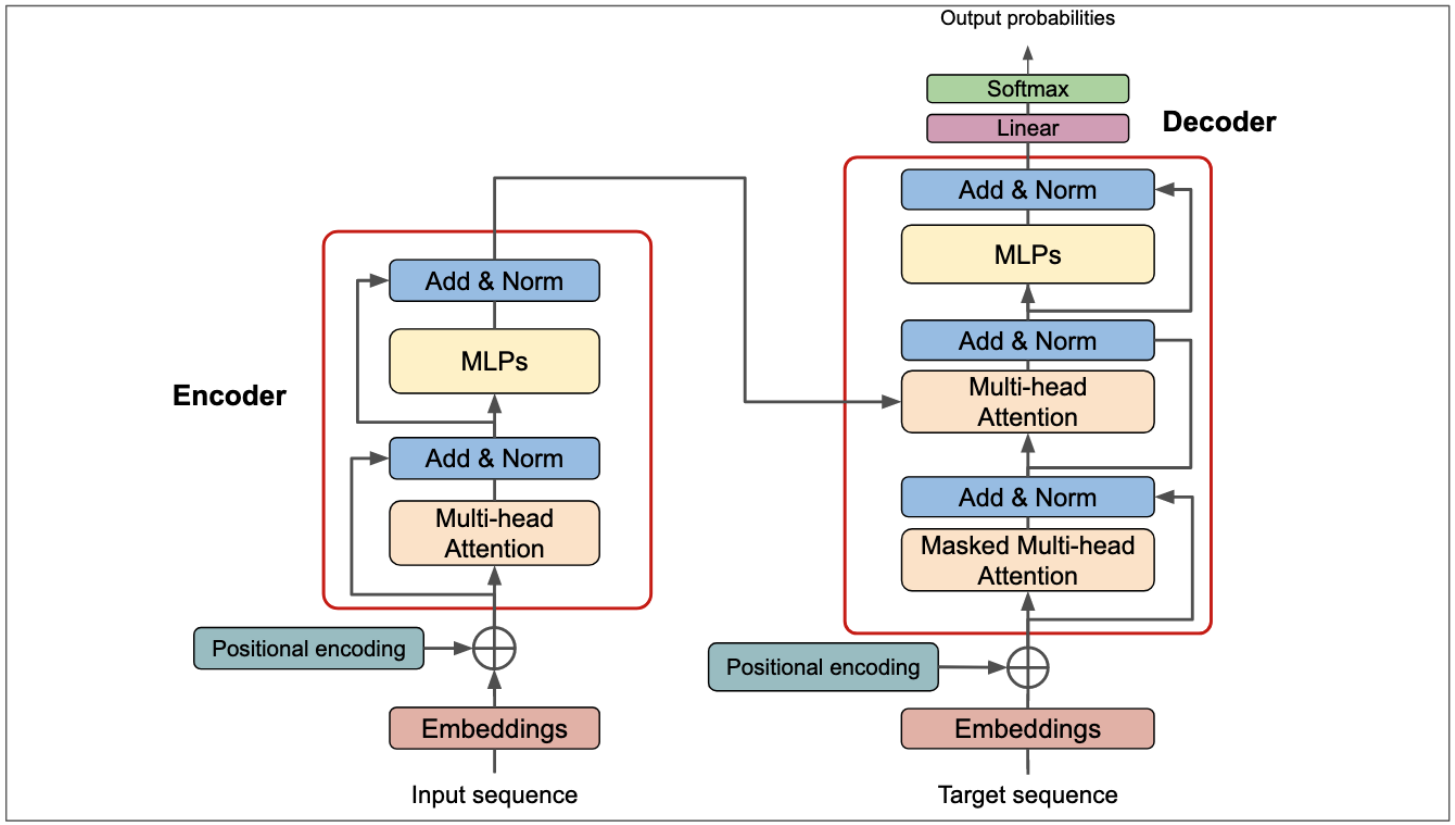
O fluxo de dados no Transformer

- O Lado do Decoder (O Escritor)
- A função do Decoder é gerar a sequência de saída, um token por vez, usando o contexto do Encoder.
- **Entrada:** O Decoder recebe a *Target sequence* (a saída esperada), que também passa por **Embeddings** e **Positional Encoding**.
- **Atenção Mascarada:** A primeira camada de atenção é uma **Masked Multi-Head Attention**. A "máscara" impede que o modelo espie os tokens futuros da sequência ao prever o token atual, garantindo que ele não "trapaceie".
- **Atenção Encoder-Decoder:** Após uma camada **Add & Norm**, vem o passo crucial. Uma segunda camada de **Multi-Head Attention** recebe:
    - **Query (Q)** do próprio Decoder.
    - **Key (K) e Value (V)** da saída do **Encoder**. É aqui que o Decoder consulta a memória da entrada para decidir qual token gerar.
- **Processamento Final:** O restante do bloco é igual ao do Encoder: uma camada **Add & Norm**, uma rede **Feed-Forward (MLPs)** e uma última camada **Add & Norm**.



O fluxo de dados no Transformer

- A Saída Final
- **Linear:** A saída do último bloco Decoder passa por uma camada **Linear**, que a transforma em um grande vetor de scores, um para cada palavra possível no vocabulário.
- **Softmax:** A função **Softmax** converte esses scores em probabilidades.

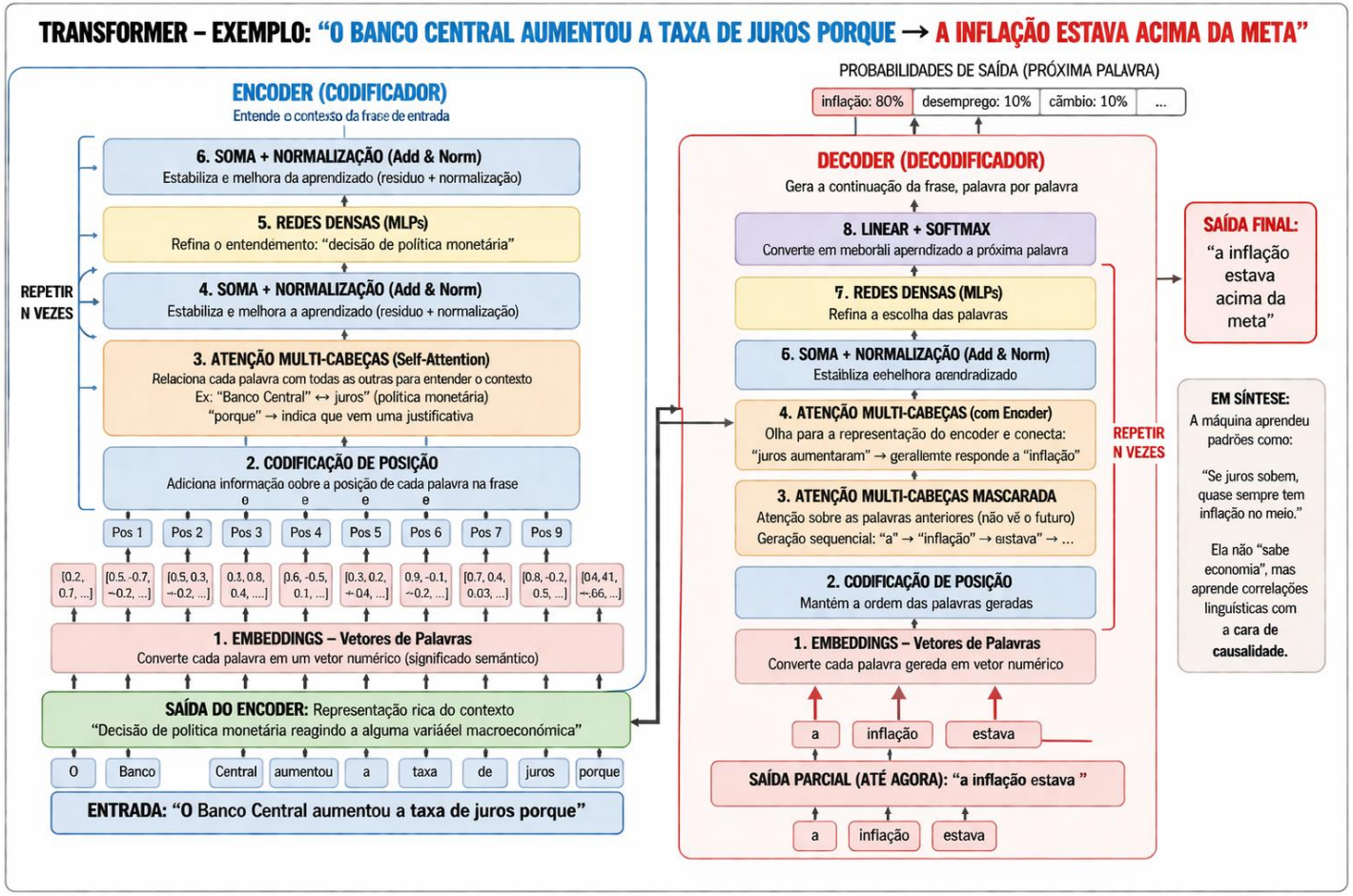


Um exemplo simples...

Frase inicial:

O Banco Central aumentou a taxa de juros porque...

Como o transformer entende...



Entrada (Encoder)

Função: entender o contexto econômico da frase

Tradução dos blocos:

- **Embeddings → Vetores de palavras**
  - “Banco Central”, “juros”, “aumentou”… viram números
  - Não é texto, é geometria
- **Positional Encoding → Codificação de posição**
  - A ordem importa: “aumentou juros” ≠ “juros aumentaram o Banco Central” (caos macroeconômico)
- **Multi-head Attention → Atenção multi-cabeças**
  - O modelo pergunta:
  - “o que importa mais aqui?”
  - Ex:
    - “Banco Central” ↔ “juros” → política monetária
    - “porque” → espera causalidade
- **Add & Norm → Soma + normalização**
  - Estabiliza o aprendizado (tipo Copom tentando não perder o controle)
- **MLPs → Redes densas**
  - Refinam o entendimento:
    → “isso aqui é sobre decisão de política monetária”

Saída do Encoder

Uma representação rica do contexto:

- “Estamos falando de política monetária reagindo a alguma variável macro”

Decodificador (Decoder)

Função: gerar a continuação da frase

Tradução dos Blocos

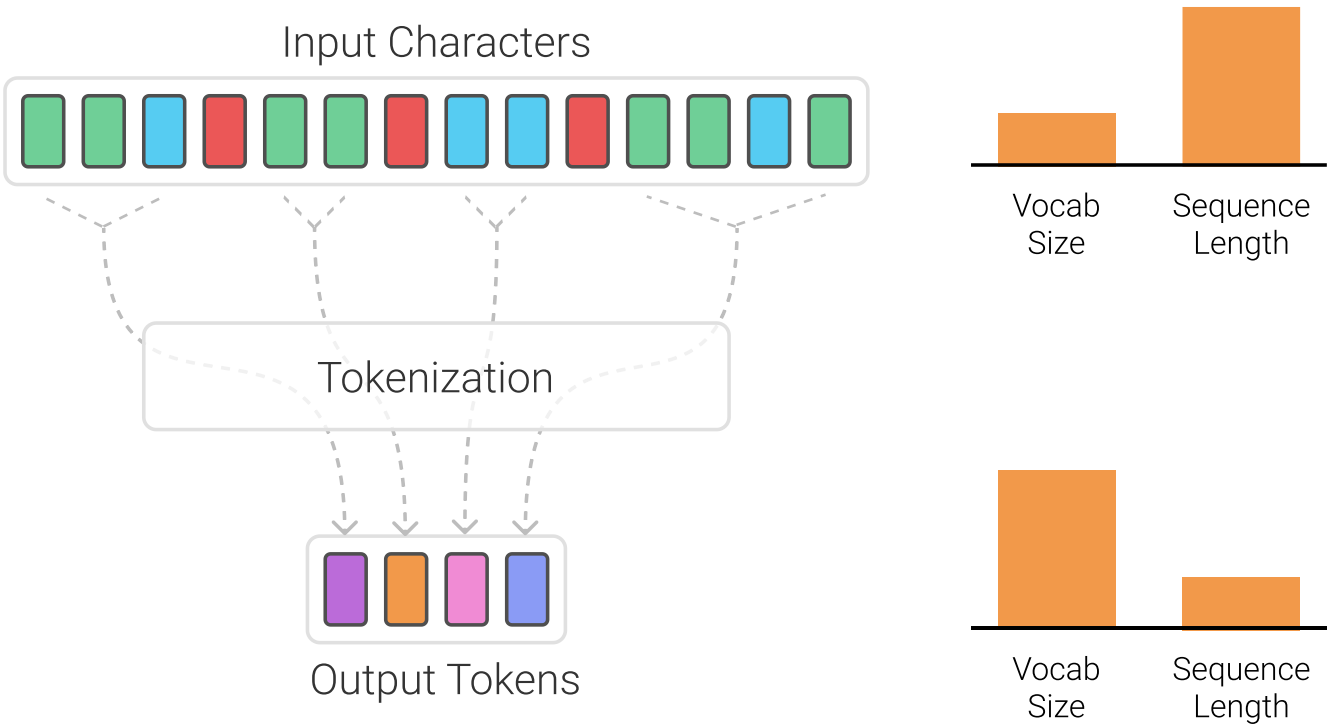
- **Decoder passo a passo**
- **Embeddings (da saída parcial)**
  - Começa com algo como:
  - “a inflação…”
- **Masked Multi-head Attention → Atenção mascarada**
  - Só olha para o passado (não pode prever o futuro olhando o gabarito)
  - Geração é sequencial:
    - “a” → “inflação” → “estava” → …
- **Multi-head Attention (com encoder)**
  - Aqui acontece a mágica:
    - O decoder olha para o encoder e pergunta:
    - “qual contexto explica isso?”
  - Ele conecta:
    - “juros aumentaram” → normalmente responde a inflação
- **MLPs → refinamento**
  - Ajusta a escolha das palavras
- **Linear + Softmax**
  - Converte em probabilidades:
    - inflação: 80%
    - desemprego: 10%
    - câmbio: 10%

Saída final (Output)

A inflação estava acima da meta

Vamos de novo?

O mundo vira pedaços...



A frase não entre inteira no modelo, ela vira *tokens*...

- ["O", "Banco", "Central", "aumentou", "a", "taxa", "de", "juros", "porque"]

Tokens viram vetores...

- Cada token vira um vetor:

"juros" → [0.21, -0.88, 0.47, ...]

Palavras semelhantes ficam próximas...

- “Juros” = “taxa”
- “inflação” = “preços”

- O modelo transforma linguagem em *geometria*.

Atenção

- Quando o modelo lê:

“aumentou a taxa de juros porque...”

Ele pergunta:

“Para prever o próximo token, em que eu devo prestar atenção?”

O “porque” olha para:

- “Aumentou”
- “Juros”
- Ignora “O”

Isso gera *pesos de atenção*.

Atencão = relevância contextual dinâmica.

Treinamento

- O modelo aprende assim:

Entrada: “O Banco Central aumentou a taxa de juros porque”

Resposta esperada:

“…” => “a inflação”

Erro => ajusta pesos => repete bilhões de vezes.

Não tem consciê*ncia, tem otimização*.

RLHF

- Depois do treino bruto, entra o ajuste fino:

Humanos avaliam respostas.

O modelo aprende o que é:

- Útil;
- Seguro;
- Claro.

É o *Reinforcement Learning with Human Feedback (RLHF)*.

É onde o modelo aprende a “parecer inteligente”.

Raciocínio

Quando ele responde:

“...porque a inflação estava acima da meta”.

Parece raciocínio.

Mas é:

- Padrões aprendidos;
- Cadeias de probabilidade.

O modelo não pensa. Ele simula pensamento.

Multimodalidade

- Agora o jogo sobe:

- Texto => embedding
- Imagem => embedding
- Áudio => embedding

Tudo vai para o mesmo espaço vetorial.

Multimodalidade

- O modelo consegue:

- Descrever imagens;
- Ler gráficos;
- Gerar texto a partir de áudio.

Contexto

- O modelo só enxerga o que está na janela.

Exemplo:

[limite: 8k tokens]

Se passar disso, esquece.

Isso não é memória real. É só contexto temporário.

Memória – como contornar

Estratégias usadas na prática:

- RAG (buscar documentos externos)
- Resumos iterativos

Você dá memória artificial ao modelo.

Limitações de janela

- Problemas reais:

- Perde contexto antigo;
- Esquece instruções;
- Respostas inconsistentes.

O modelo é brilhante. Mas sofre de amnésia seletiva.

Estratégias de contorno

- Chunking (quebrar texto)
- Embedding + busca
- Chain of thought
- Agentes (loop de decisões)

Hype atual: agentes de IA

Conclusão

- Transformers não entendem o mundo. Eles entendem padrões. E, surpreendentemente, isso é suficiente para parecer inteligência.